

WPA Laboratorium 8 -- Matplotlib

Instrukcja

Zadania i przykłady proszę wykonywać równoległe z tekstem. Środowisko jest dowolne chyba, że wskazano inaczej.

Wprowadzenie

Matplotlib jest biblioteką, która dostarcza zestaw narzędzi pozwalający na tworzenie:

- wykresów 2D/3D rozmaitych typów
- w ograniczonym zakresie: animacji

Jest to biblioteka:

- wieloplatformowa
- popularna
- produkująca grafiki „production-quality” w wielu formatach

Strona www: <https://matplotlib.org>

UWAGA

- matplotlib to nie jest standardowy moduł Pythona - trzeba go zainstalować w środowisku
- po szczęśliwym zainstalowaniu:

```
# wystarczająco dużo funkcjonalności na początek
import matplotlib.pyplot as pyplot # kanoniczny sposób 1
```

```
# ALBO
```

```
# wystarczająco dużo funkcjonalności na początek
import matplotlib.pyplot as plt    # kanoniczny sposób 2
```

Dodatkowe funkcjonalności dostępne są w podmodułach (patrz. dokumentacja biblioteki, przykłady)

Demonstracja możliwości oraz przykłady użycia: <https://matplotlib.org/stable/gallery/index.html>

Zadanie

Przeskorolować stronę z przykładami.

Quick start

UWAGA : W dalszej części zakładamy, że biblioteka została zaimportowana w następujący sposób

```
import matplotlib.pyplot as pyplot
```

Zadanie: Hello World

Uruchomić poniższy kod w Jupyterze oraz jako skrypt.

```
# stworzenie wykresu
pyplot.plot([1,2,3,4,5],[2,-1,0,-1,2],)
            # x-values #y-values

pyplot.title('Hello World')

# obrazek został stworzony i zapamiętany

# wyświetlenie stworzonego obrazka
pyplot.show()
```

- Zakomentować `pyplot.show()`. Co się stało?
- Uwaga: w dalszej części `pyplot.show()` będzie pomijane.
- można pominąć punkty na osi x, sprawdź `pyplot.plot([2,-1,0,-1,2])`

Line Plot

```
# Set up the data
x = [0, 1, 2, 3, 4, 5, 6]
y = [0, 2, 6, 14, 30, 43, 75]

# Set the axes headings
pyplot.ylabel('Speed',      fontsize=12)
            # axis label # ?

pyplot.xlabel('Time', fontsize=12)

# Set the title
pyplot.title("Speed v Time")

# Plot and display the graph
# Using blue circles for markers ('bo')
# and a solid line ('-')
pyplot.plot(x,      y,      'bo-')
            # x-values # y-values # line style
```

Jeden ze sposobów umieszczenia dwóch wykresów na jednym obrazku:

```
x1 = [0, 1, 2, 3, 4, 5, 6]
y1 = [0, 2, 6, 14, 10, 13,15]

pyplot.plot(x1,y1)

x2 = [0, 0.5, 1, 2.5, 4, 5, 6]
y2 = [1, 4, 5, 1, 3, 4, 7]

pyplot.plot(x2,y2)
```

Na wygląd wykresu składa się:

- kolor
- styl linii

- marker
- grubość linii
- inne

Parametry określające wygląd mogą być podane w formie skróconej [color][marker][line] lub jawnie:

```
pyplot(x,y, 'bo-')

# is equivalent to

pyplot.plot(x,y,
    color = 'b',
    marker = 'o',
    linestyle = '- '
)
```

Kolory, color

Skrócone nazwy kolorów:

```
'b' as blue
'g' as green
'r' as red
'c' as cyan
'm' as magenta
'y' as yellow
'k' as black
'w' as white
```

Kolory można także podawać w formie RGB (r,g,b), nazwą indigo, teal, brown i na inne sposoby (w tym dodanie przezroczystości)

- [Specifying colors](#)
- [List of named colors](#)

Style linii, linestyle

character	description
'-'	solid line style
'--'	dashed line style
'-.'	dash-dot line style
':'	dotted line style

Markery, marker

Wybrane markery:

character	description
'.'	point marker
'o'	circle marker
'v'	triangle_down marker
'^'	triangle_up marker
's'	square marker
'p'	pentagon marker

'*'	star marker
'+'	plus marker
'x'	x marker
'D'	diamond marker

Inne parametry

- linewidth -- grubość lini w pt
- markersize
- markerfacecolor,
- markeredgewidth, markeredgewidth

Zadania

Stworzyć obrazek przedstawiający 4 wykresy funkcji postaci $y = x + n$. Każdy z tych wykresów musi mieć inny styl.

Czy umiesz tak zmodyfikować program tak, aby to użytkownik określił jak ma wyglądać wykres?

- poproś o podanie kolorów, a potem markerów
- poproś o podanie kolejno formatu dla każdego obrazka

Pełna dokumentacja funkcji `pyplot.plot`: [matplotlib.pyplot.plot](https://matplotlib.org/3.1.1/api/_as_gen/matplotlib.pyplot.plot.html)

Scatter plot

```
# Create data
riding = [[17, 18, 21, 22, 19, 21, 25, 22, 25, 24],[3, 6, 3.5, 4, 5, 6.3, 4.5, 5, 4.5, 4]]
swimming = [[17, 18, 20, 19, 22, 21, 23, 19, 21, 24],[8, 9, 7, 10, 7.5, 9, 8, 7, 8.5, 9]]
sailing = [[31, 28, 29, 36, 27, 32, 34, 35, 33, 39],[4, 6.3, 6, 3, 5, 7.5, 2, 5, 7, 4]]

# Plot the data
pyplot.scatter(x=riding[0], y=riding[1], c='red', marker='o', label='riding')

pyplot.scatter(x=swimming[0], y=swimming[1], c='green', marker='^', label='swimming')

pyplot.scatter(x=sailing[0], y=sailing[1], c='blue', marker='*', label='sailing')

# Configure graph
pyplot.xlabel('Age')

pyplot.ylabel('Hours')

pyplot.title('Activities Scatter Graph')

pyplot.legend()
```

Zadanie

Zmienić kolory i markery na wykresie otrzymanym w poprzednim przykładzie.

Bar chart

```
# Set up the data
labels = ('Python', 'Scala', 'C#', 'Java', 'PHP')
index = (1, 2, 3, 4, 5) # provides locations on x axis
sizes = [45, 10, 15, 30, 22]

# Set up the bar chart
pyplot.bar(index, sizes, tick_label=labels)

# Configure the layout
pyplot.ylabel('Usage')
pyplot.xlabel('Programming Languages')
```

Zadanie

Funkcja `pyplot.barh()` tworzy poziomy *bar chart*. Przetestować, używając danych z poprzedniego przykładu.

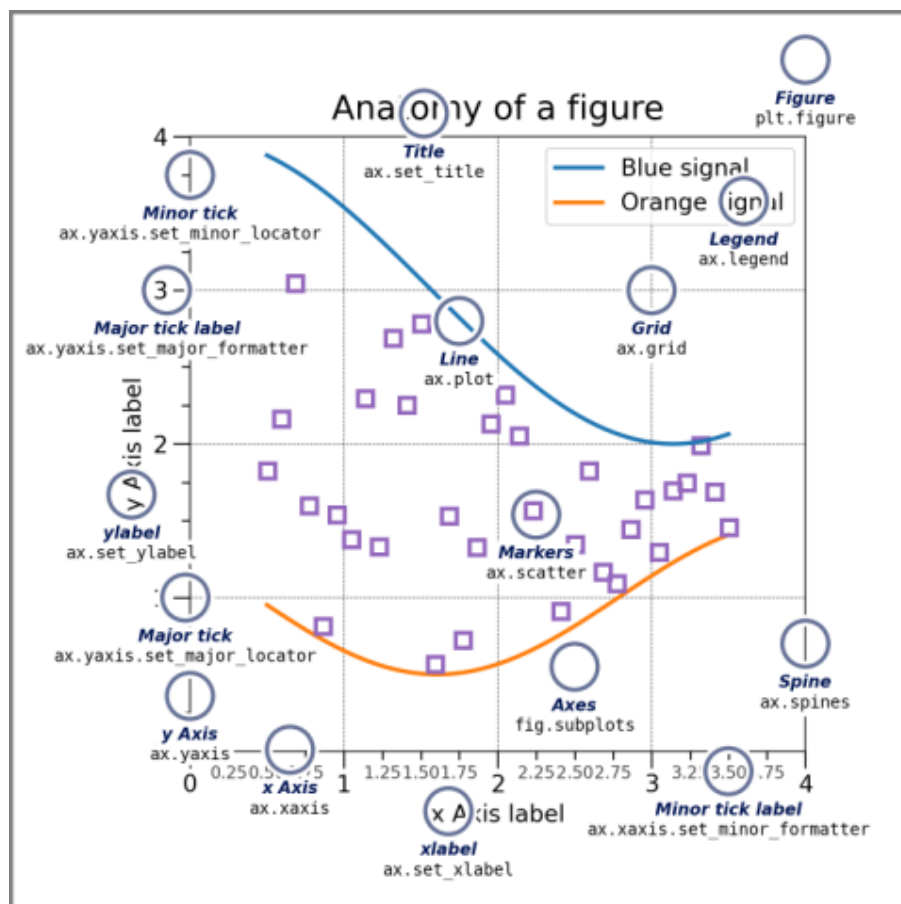
Zadanie

Co się stanie?

```
pyplot.bar(x_values, sizes, tick_label=labels, color=('red', 'green', 'blue', 'yellow', 'orange'))
```

Figures and Subplots

A matplotlib figure is the object that contains all the graphical elements displayed on a plot: the axes, the legend, the title, lines, bars, etc. Each element can be adapted to our needs.



The `matplotlib.pyplot.figure()` function generates a figure object. This function returns a `matplotlib.figure.Figure` object. It is then possible to interact directly with the figure object. For example it is possible to add axes to the figure, to add sub plots to a graph etc.

Working directly with the figure is necessary if you want to add multiple subplots to a figure. This can be useful if what is required is to be able to compare different views of the same data side by side.

Each subplot has its own axes which can coexist within the figure. One or more subplots can be added to a figure using the `figure.add_subplot()` method.

A subplot can be added using a 3-digit integer (or three separate integers) describing the position of the subplot. The digits represent the number of rows, columns and the index of the sub plot within the resulting matrix.

```
# run in jupyter and as a script
figure_1 = pyplot.figure()

sub_plot_2_2_1 = figure_1.add_subplot(221)
sub_plot_2_2_1.set(title='(1,2,2)')

sub_plot_2_2_2 = figure_1.add_subplot(222)
sub_plot_2_2_2.set(title='(2,2,2)')

sub_plot_2_2_3 = figure_1.add_subplot(223)
sub_plot_2_2_3.set(title='(2,2,3)')

sub_plot_2_2_4 = figure_1.add_subplot(224)
sub_plot_2_2_4.set(title='(2,2,4)')
```

Adding plots:

```
x = range(0,10)
y = range(-5,5)

figure_1 = pyplot.figure()

sub_plot_2_2_1 = figure_1.add_subplot(221)
sub_plot_2_2_1.set(title='(1,2,2)')

sub_plot_2_2_1.plot(x,y,'rD:')
sub_plot_2_2_1.plot(x,[i**2 for i in y],'co-')

sub_plot_2_2_2 = figure_1.add_subplot(222)
sub_plot_2_2_2.set(title='(2,2,2)')
sub_plot_2_2_2.scatter([x,y],[y,x])

sub_plot_2_2_3 = figure_1.add_subplot(223)
sub_plot_2_2_3.set(title='(2,2,3)')
sub_plot_2_2_3.bar(x,y)
sub_plot_2_2_3.bar(y,x)

sub_plot_2_2_4 = figure_1.add_subplot(224)
sub_plot_2_2_4.set(title='(2,2,4)')
sub_plot_2_2_4.barh(x,y)
```

Zadanie

The following table provides information on cities in the UK and their populations (note that London has been omitted as its population is so much larger than that of any other city and this would distort the graph).

City	Population
Bristol	617,280
Cardiff	447,287
Bath	94,782
Liverpool	864,122
Glasgow	591,620
Edinburgh	464,990
Leeds	455,123
Reading	318,014
Swansea	300,352
Manchester	395,515

Using this data create one figure with two subplots:

1. A scatter plot for the city to population data.
2. A bar chart for the city to population data.
3. Customize both.

Zadania

Zadanie 1.

Biblioteka matplotlib umożliwia również tworzenie „pie chart”. Odnaleźć w dokumentacji lub internetach stosowną funkcję i pobawić się nią i jej opcjami.

Zadanie 2.

Przeanalizować kod:

```
# Set up the data
labels = ('Python', 'Scala', 'C#', 'Java', 'PHP')
index = (1, 2, 3, 4, 5)

web_usage = [20, 2, 5, 10, 14]

data_science_usage = [15, 8, 5, 15, 2]

games_usage = [10, 1, 5, 5, 4]

web_and_games_usage = [web_usage[i] + data_science_usage[i] for i in range(0, len(web_usage))]

# Set up the bar chart
pyplot.bar(index, web_usage, tick_label=labels, label='web')

pyplot.bar(index, data_science_usage, tick_label=labels, label='data science', bottom=web_usage)

pyplot.bar(index, games_usage, tick_label=labels, label='games', bottom=web_and_games_usage)

# Configure the layout
pyplot.ylabel('Usage')
pyplot.xlabel('Programming Languages')
pyplot.legend()
```

- Dodać kolejny (wymyślony) język programowania

- Dodać kolejne pole zastosowań

Zadanie 3.

Przeanalizować kod

```
BAR_WIDTH = 0.15

# set up grouped bar charts
teama_results = (60, 75, 56, 62, 58)
teamb_results = (55, 68, 80, 73, 55)

# Set up the index for each bar
index_teama = (1, 2, 3, 4, 5)
index_teamb = [i + BAR_WIDTH for i in index_teama]

# Determine the mid point for the ticks
ticks = [i + BAR_WIDTH / 2 for i in index_teama]

tick_labels = ('Lab 1', 'Lab 2', 'Lab 3', 'Lab 4', 'Lab 5')

# Plot the bar charts
pyplot.bar(index_teama, teama_results, BAR_WIDTH, color='b', label='Team A')
pyplot.bar(index_teamb, teamb_results, BAR_WIDTH, color='g', label='Team B')

# Set up the graph
pyplot.xlabel('Labs')
pyplot.ylabel('Scores')
pyplot.title('Scores by Lab')
pyplot.xticks(ticks, tick_labels)
pyplot.legend()

# Display the graph
pyplot.show()
```

- Dodać kolejną drużynę
- Dodać kolejny Lab

Uzupełnienia

Oczekiwany typ danych

Matplotlib oczekuje, że dane przekazywane do wykresów będą typu `numpy.array` lub podobnego. Jest to typ pochodzący z bardzo rozbudowanej biblioteki [numpy](#).

Matplotlib akceptuje również typy, które potrafi przekonwertować. W naszym przypadku działa się to gdy jako argument był przekazywany typ `list` albo `tuple`.

Na nasze potrzeby wystarczy schemat podany niżej

```
# kanoniczny sposób
import numpy as np
import matplotlib.pyplot as pyplot

# linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None, axis=0)
# Return evenly spaced numbers over a specified interval.
# check
# help(np.linspace)

x_data = np.linspace(-10,10)
coarse_data = np.linspace(-10,10,6)

# try it
# print(x_data)
# print(coarse_data)

pyplot.plot(x_data,[x**2 for x in x_data],label='fine')
pyplot.plot(coarse_data,[x**2 for x in coarse_data],label='coarse')

pyplot.legend()

pyplot.show()
```

Inny schemat

```
import math

raw_data_1 = np.matrix( [ range(0,10), [math.sin(x) for x in range(0,10)] ] )
data_1 = np.asarray(raw_data_1)

# check help(np.random.randint)
raw_data_2 = np.matrix( [ np.linspace(0,10,20), np.random.randint(-5,6,20) ] )
data_2 = np.asarray(raw_data_2)

# print both raw_data and data, check type

pyplot.scatter(data_1[0],data_1[1])
pyplot.plot(data_1[0], [ math.cos(x) for x in data_1[0] ] )
pyplot.scatter(data_2[0],data_2[1])
```

Na niedosyt, proszę zwrócić uwagę na wykorzystanie słownika `data`

```
np.random.seed(19680801) # seed the random number generator.
data = {'a': np.arange(50),
        'c': np.random.randint(0, 50, 50),
```

```

        'd': np.random.randn(50)}
data['b'] = data['a'] + 10 * np.random.randn(50)
data['d'] = np.abs(data['d']) * 100

fig, ax = plt.subplots(figsize=(5, 2.7), layout='constrained')
ax.scatter('a', 'b', c='c', s='d', data=data)
ax.set_xlabel('entry a')
ax.set_ylabel('entry b')

```

Dwa style pracy

There are essentially two ways to use Matplotlib:

pyplot-style

- Rely on pyplot to implicitly create and manage the Figures and Axes, and use pyplot functions for plotting.

```

x = np.linspace(0, 2, 100) # Sample data.

plt.figure(figsize=(5, 2.7), layout='constrained')
plt.plot(x, x, label='linear') # Plot some data on the (implicit) axes.
plt.plot(x, x**2, label='quadratic') # etc.
plt.plot(x, x**3, label='cubic')
plt.xlabel('x label')
plt.ylabel('y label')
plt.title("Simple Plot")
plt.legend("Simple Plot")
plt.legend()

```

OO-style

- Explicitly create Figures and Axes, and call methods on them (the "object-oriented (OO) style").

```

x = np.linspace(0, 2, 100) # Sample data.

# Note that even in the OO-style, we use `.pyplot.figure` to create the Figure.
fig, ax = plt.subplots(figsize=(5, 2.7), layout='constrained')
ax.plot(x, x, label='linear') # Plot some data on the axes.
ax.plot(x, x**2, label='quadratic') # Plot more data on the axes...
ax.plot(x, x**3, label='cubic') # ... and some more.
ax.set_xlabel('x label') # Add an x-label to the axes.
ax.set_ylabel('y label') # Add a y-label to the axes.
ax.set_title("Simple Plot") # Add a title to the axes.
ax.legend() # Add a legend.

```

Funkcje i matplotlib

If you need to make the same plots over and over again with different data sets, or want to easily wrap Matplotlib methods, use the recommended signature function below.

```

def my_plotter(ax, data1, data2, param_dict):
    """
    A helper function to make a graph.
    """

```

```
out = ax.plot(data1, data2, **param_dict)
return out
```

which you would then use twice to populate two subplots:

```
data1, data2, data3, data4 = np.random.randn(4, 100) # make 4 random data sets

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(5, 2.7))

my_plotter(ax1, data1, data2, {'marker': 'x'})
my_plotter(ax2, data3, data4, {'marker': 'o'})
```

Zapisywanie do pliku

Biblioteka `matplotlib` korzysta z tzw. *backend* do renderowania obrazków. Domyślnie jest to *backend* interaktywny. Proszę zwrócić uwagę, że w Jupyterze jest inny niż gdy uruchamiamy ten sam program jako skrypt.

Dostępne są też *backends* które pozwalają zapisywać obrazki do pliku.

```
# jeden ze sposobów
pyplot.plot(x, [i**2 for i in x], label='quadratic')
pyplot.title("Simple Plot") # Add a title to the axes.
pyplot.legend() # Add a legend.

# TRY IT
matplotlib.pyplot.savefig('my_figure', backend='pdf', format='pdf')
# matplotlib.pyplot.savefig('next_figure.svg', backend='svg', format='svg')
# matplotlib.pyplot.savefig('next_figure.png', backend='agg', format='png')

# drugi ze sposobów

# good practice
original_backend = matplotlib.get_backend()

# TRY IT
matplotlib.use('pdf')
# matplotlib.use('svg')
# matplotlib.use('agg')

pyplot.plot(x, [i**2 for i in x], label='quadratic')
pyplot.title("Simple Plot") # Add a title to the axes.
pyplot.legend() # Add a legend.

matplotlib.pyplot.savefig('my_figure')

# good practice
matplotlib.use(original_backend)
```

Więcej informacji (techniczne): [Matplotlib Backends](#)

Zadania

Zadanie 1.

Napisać program, który

- pobiera od użytkownika końce przedziału $[a, b]$
- tworzy pliki png zawierające wykresy kilku funkcji na tym przedziale,
- nazwy plików powinny składać się z nazwy funkcji oraz końców przedziału

Zadanie 2.

Przyjrzyć się dokładniej i poszukać inspiracji: [Matplotlib examples](#)

Zadanie 3.

Ściągnąć notebook Jupytera na dole strony i prześledzić : [Matplotlib quick star](#)

Zadanie 4.

Szukać pomocy w razie potrzeby: [Matplotlib tutorials](#)